

极限编程（eXtreme Programming, XP）笔记

基于 Thomas Dudziak 1999 年《eXtreme Programming An Overview》整理

2026 年 5 月 15 日

目录

1 概述	2
1.1 什么是极限编程（XP）	2
1.2 文档结构	2
2 基本概念	2
2.1 四个核心变量	2
2.2 四个核心价值观	3
2.3 变更驱动流程	3
3 XP 流程	4
3.1 规划游戏（Planning Game）	4
3.1.1 探索阶段（Exploration）	4
3.1.2 规划阶段（Planning）	4
3.1.3 指导阶段（Steering）	4
3.2 迭代开发	4
4 XP 方法论：规则与实践	5
4.1 管理规则与实践	5
4.1.1 核心指标	5
4.1.2 角色定义	6
4.1.3 其他管理实践	6
4.2 开发规则与实践	6
4.2.1 开发周期（单任务）	6
4.2.2 核心开发实践	6
4.3 设计规则与实践	7
4.3.1 核心设计原则	7
4.3.2 设计辅助方法	7
5 结论	8
6 参考文献	8

1 概述

1.1 什么是极限编程 (XP)

极限编程 (XP) 既是软件开发过程，也是方法论和可定制的过程框架，属于 Alistair Cockburn 提出的水晶方法论 (Crystal Methodology) 家族，具有以下特点：

- 轻量级：高生产力、高容错性
- 强沟通：以非正式、短路径沟通为主
- 少交付物：仅保留必要产出，但发布频率高
- 精简角色：仅定义少量核心角色和活动

适用范围：

- 业务类项目（有明确外部客户）
- 项目周期：6-15 个月
- 团队规模：2-12 人（核心限制条件）

不适用场景：需求可提前完全确定的项目。

1.2 文档结构

本文档分为四个核心部分：

1. 基本概念：四个变量、四个价值观、变更驱动流程
2. XP 流程：规划游戏、迭代开发机制
3. XP 方法论：管理、开发、设计三类规则与实践
4. 结论与核心参考文献

2 基本概念

2.1 四个核心变量

XP 将软件项目视为由四个可控制变量组成的系统，其中**时间是核心变量**，所有其他变量的变化最终都会影响时间。

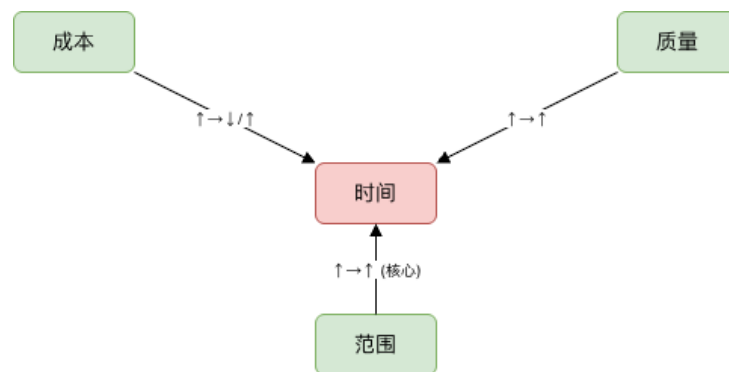
表 1: XP 四个核心变量及其关系

变量	定义	与时间的关系
成本	项目投入的资金、人力、设备等资源	成本 ↑ → 时间 ↓（后期加人可能反增）；成本 ↓ → 时间大幅 ↑
时间	系统/发布的交付截止日期	核心变量，其他变量的变化最终影响时间
质量	系统正确性（客户定义）和测试覆盖率	质量 ↑ → 时间 ↑；质量 ↓ → 时间有限度减少
范围	系统需要实现的功能数量和复杂度	范围 ↑ → 时间 ↑；范围 ↓ → 时间 ↓（核心控制关系）

变量控制规则：

- 业务方（管理层 + 客户）：控制**成本、质量**，并在**时间/范围**中二选一

- 开发方：控制剩余的一个变量
- 禁止场景：同时固定时间和范围（无法通过成本和质量有效控制）



业务方控制：成本、质量 + 时间/范围二选一
开发方控制：剩余变量

图 1：四个变量关系图

2.2 四个核心价值观

所有 XP 实践都围绕这四个价值观展开，它们是 XP 的灵魂：

1. **沟通**：项目失败多源于沟通中断；代码和文档也是重要的沟通方式
2. **简单**：系统和方法论都保持最简；仅保留必要交付物（用户故事、计划、代码）
3. **反馈**：全流程反馈机制（单元测试即时反馈、功能测试、频繁发布）；反馈越早，修复成本越低
4. **勇气**：敢于反传统软件工程实践；支持客户深度参与；无情重构代码

2.3 变更驱动流程

XP 的核心设计目标是应对软件项目中最常见的风险——**需求变更**。

软件项目主要风险：

- 需求变更：用户需求变化、IKIWISI 效应（"I'll know it when I see it"）、市场变化、需求不精确
- 技术风险：新技术或快速变化的技术带来的不确定性
- 性能风险：系统响应时间、负载能力不满足要求

XP 应对策略：拥抱变更

- 客户作为开发团队的正式成员
- 频繁发布（1-3 个月一个版本），让客户尽早看到系统
- 客户编写用户故事定义需求
- 团队诚实估算，让客户做范围/时间决策
- 流程设计支持任意阶段变更方向

3 XP 流程

XP 是典型的**迭代增量式**开发流程，整体结构如图 2 所示。

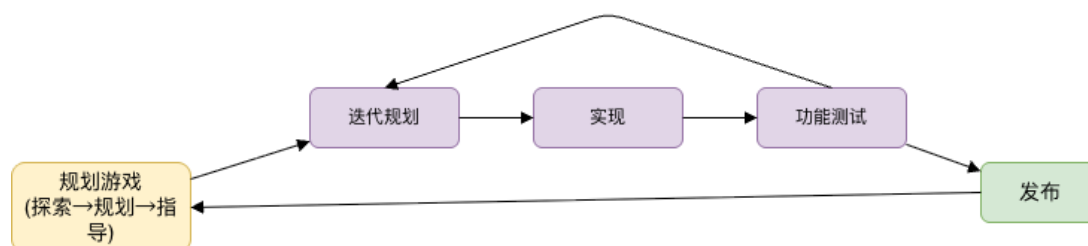


图 2: XP 简化流程结构图

3.1 规划游戏 (Planning Game)

规划游戏是 XP 中业务方和开发方协作的核心机制，分为三个阶段：

3.1.1 探索阶段 (Exploration)

- 客户编写**用户故事 (User Stories)**：写在索引卡上，1-3 周理想工程时间 (Ideal Engineering Time, IE)，可测试，避免技术细节
- 开发者估算每个故事的 IE 时间和实现风险
- 不确定问题通过 **Spike Solution (小原型)** 解决，同时校准初始负载因子

3.1.2 规划阶段 (Planning)

- 客户将故事分为三堆：必要功能、高价值功能、可选功能
- 开发者提供**负载因子 (Load Factor)**：日历时间/理想工程时间，初始值通常为 2-3
- 协商承诺计划（二选一）：
 1. 客户指定发布日期，开发者确定可实现的功能范围
 2. 客户指定功能范围，开发者确定发布日期

3.1.3 指导阶段 (Steering)

在整个发布周期内，通过以下四种”动作”调整计划：

1. 迭代：按计划执行开发
2. 恢复：若无法完成计划，重新协商发布日期或范围
3. 新故事：客户可添加新故事，需重新协商计划
4. 重新估算：若计划不准确，重新估算剩余故事并协商

3.2 迭代开发

每个发布周期由多个迭代组成，每个迭代周期为 **2-3 周**。迭代同样包含规划、执行、验证三个步骤：

1. **迭代规划**：客户选择本次迭代的故事（最有价值 + 最有风险），开发者拆分为工程任务（0.5-3 IE 天）

2. **任务认领**：开发者自愿认领任务并估算，平衡工作量
3. **开发执行**：按 XP 开发周期实现任务
4. **验证**：运行功能测试，失败的故事进入下一个迭代
5. **特殊情况**：必要时安排**重构迭代**（无新功能，仅优化代码结构）

4 XP 方法论：规则与实践

XP 的核心优势在于**所有实践相互支持**，单个实践的弱点被其他实践覆盖，形成一个完整的闭环系统。

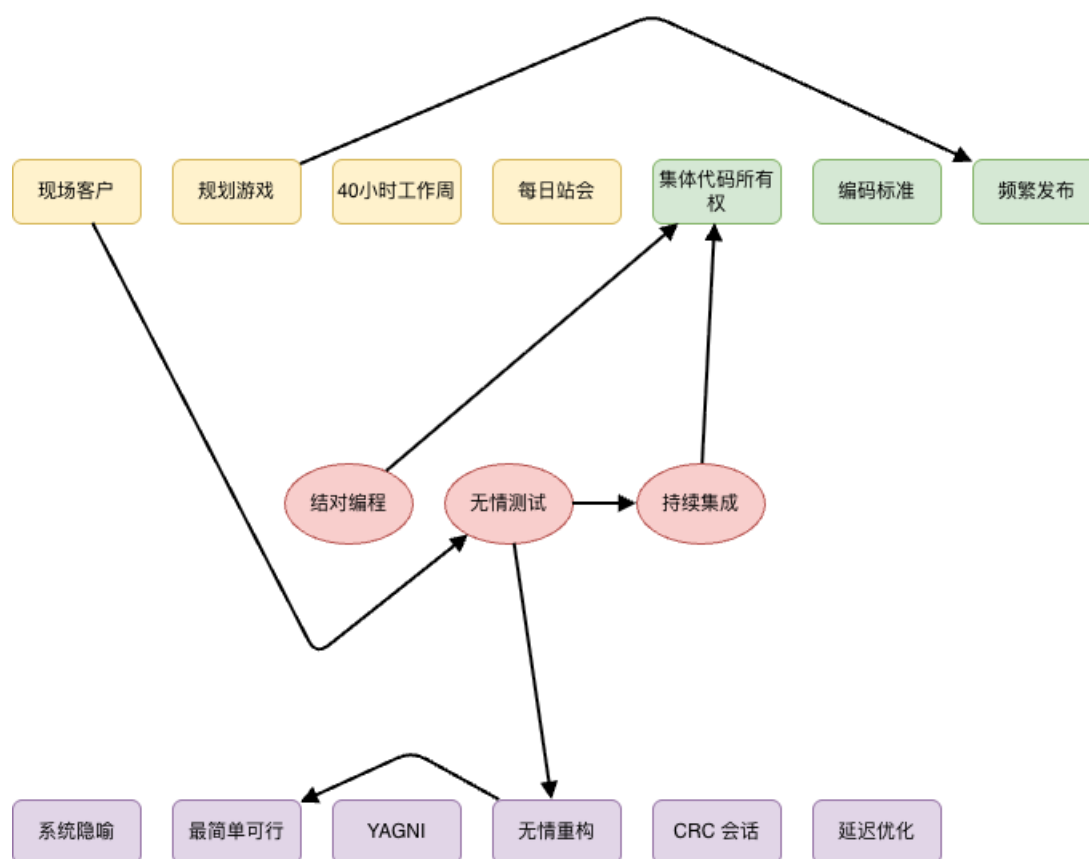


图 3: XP 规则与实践支持关系图

4.1 管理规则与实践

4.1.1 核心指标

XP 仅定义两个核心指标来衡量项目进度：

1. **负载因子**：团队速度 = 日历时间 / 理想工程时间
 - 优秀 XP 团队：2.5（2.5 个日历天完成 1 个理想工程天的工作）
 - 教练：10（大量时间用于指导和沟通）
2. **功能测试得分**：衡量项目进度，由客户决定是否接受

4.1.2 角色定义

XP 定义了六个核心角色，没有专门的分析师、架构师或设计师：

表 2: XP 核心角色与职责

角色	主要职责
客户	编写用户故事、确定优先级、编写功能测试； 现场客户 （全程参与团队）
程序员	全栈开发；编写单元测试、重构代码；无专门的技术分工
教练	技术指导、流程培训、协调团队；不做所有决策，引导团队自己做决定
跟踪者	收集负载因子和功能测试得分；判断进度是否可达成
测试者	帮助客户编写和运行功能测试（尤其是无法自动化的测试）
顾问	提供专业知识；团队学习后重写解决方案，确保知识共享

4.1.3 其他管理实践

- **工作空间**：大房间（bullpen），中间放电脑，侧边设小隔间用于讨论；专用集成机
- **每日站会**：固定时间（如 10 点），每人 1-2 分钟汇报进展和问题
- **40 小时工作周**：避免加班，保证团队精力和动力；最多偶尔加班 1-2 次

4.2 开发规则与实践

4.2.1 开发周期（单任务）

每个工程任务都遵循以下五步开发周期，整个过程通常在几小时到一天内完成：

1. 分析任务和用户故事，必要时召开 CRC 会话
2. **先写单元测试**：定义任务完成标准
3. 写最简单可行的代码使测试通过
4. 重构代码优化结构
5. 集成到代码库（单元测试必须 100% 通过）

4.2.2 核心开发实践

1. **持续集成**：任务完成且单元测试通过立即集成（至少每天一次）
 - 好处：集成简单、问题定位快、代码库始终最新
2. **集体代码所有权**：团队拥有所有代码，任何人可修改任何部分
 - 依赖：重构、持续集成、编码标准
3. **结对编程**：所有生产代码由两人共同编写（司机 + 导航）
 - 频繁换角色和配对；知识共享、代码质量更高
 - 研究表明：仅多花 10% 时间，但代码缺陷显著减少

4. **编码标准**：统一命名和格式，保证代码一致性

5. **无情测试**

- 单元测试：开发者写，先写测试再写代码；自动化；作为重构的安全网
- 功能测试：客户写，验证需求；至少每次迭代运行一次

4.3 设计规则与实践

XP 与传统软件工程最大的区别是**不做前期大设计 (Big Design Up Front)**，而是通过以下实践逐步演化出优秀设计。

4.3.1 核心设计原则

1. **DTSTTCPW**：做最简单可行的事 (Do The Simplest Thing That Could Possibly Work)
2. **YAGNI**：不实现不需要的功能 (You Are Not Gonna Need It)；删除无用代码
3. **无情重构**：在不改变外部行为的前提下优化代码结构
 - 依赖：全面的单元测试
 - 目标：代码更简单、易维护

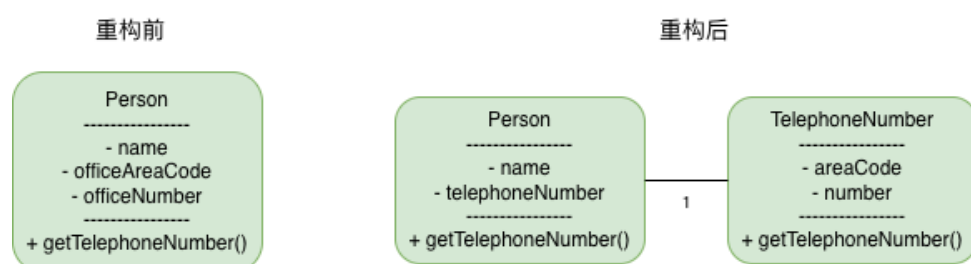


图 4：提取类重构示例

4.3.2 设计辅助方法

1. **CRC 会话**：Class-Responsibility-Collaborator（类-责任-协作者）
 - 用索引卡表示类，左边写责任，右边写协作者
 - 动态讨论设计，不做永久文档
 - 通过”走查”用户故事验证设计
2. **系统隐喻**：用统一的业务隐喻指导设计和命名
 - 示例：C3 工资系统使用”制造隐喻”（生产线、零件、仓库、工作站）
 - 好处：改善沟通，让客户和开发者有共同语言
3. **延迟优化**：先让系统工作，再优化性能；通过测量找到瓶颈再优化

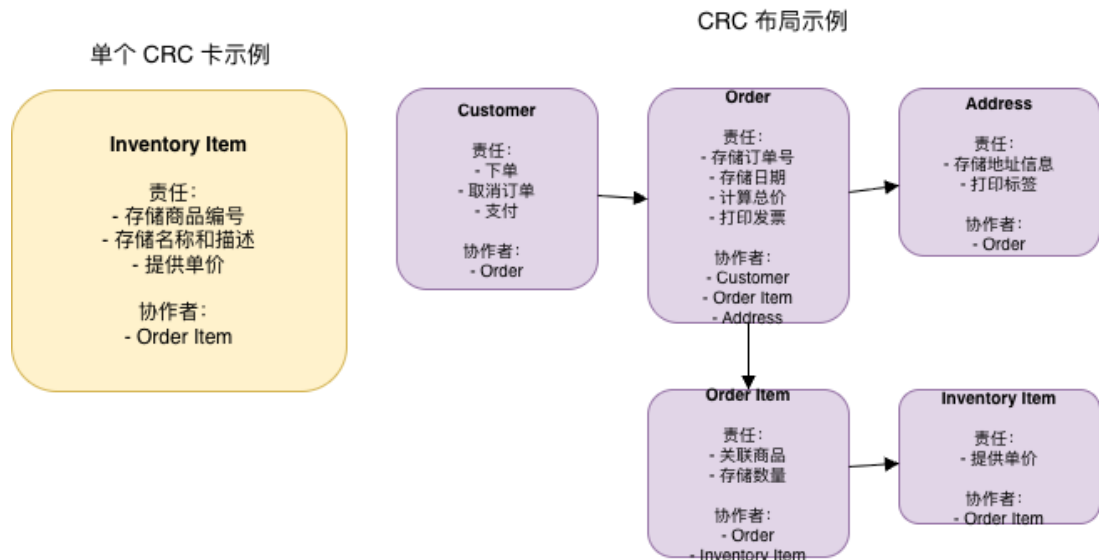


图 5: CRC 卡与布局示例

5 结论

XP 的核心力量在于**所有实践的协同作用**，单个实践无法发挥最大效果。与传统软件工程相比，XP 具有以下特点：

- 拥抱变更：将需求变更视为正常现象，而非需要避免的问题
- 客户深度参与：现场客户全程参与开发，确保系统符合业务需求
- 迭代增量：频繁发布，快速获得反馈
- 团队导向：强调沟通、协作和知识共享

主要优势：

- 适应需求变化能力强
- 团队知识共享充分
- 代码质量高，缺陷少
- 开发过程更有趣，团队士气高

局限性：

- 仅适用于小团队（2-12 人）
- 需要客户全程深度参与
- 不适用于需求可提前完全确定的项目

6 参考文献

参考文献

- [1] Kent Beck. *eXtreme Programming explained –Embrace Change*. Addison-Wesley, 1999.

- [2] Martin Fowler. *Refactoring –Improving the Design of Existing Code*. Addison-Wesley, 1999.
- [3] David Bellin, Susan Suchman Simone. *The CRC Card Book*. Addison-Wesley, 1997.
- [4] Chrysler Goes To "Extremes" –Case Study. Distributed Computing, October 1998.
- [5] Kent Beck. Embracing Change with Extreme Programming. *IEEE Computer*, Vol.32, No.10, October 1999.
- [6] Alistair Cockburn & Laurie Williams. The Costs and Benefits of Pair Programming. 2000.